IPO-Ready | Target product architecture

C4 container view: a financial intelligence platform for discovering, comparing, evaluating and monitoring companies.

BLUE: application / worker

TEAL: trusted data / serving

GOLD: external dependency

DASHED: optional extension

IPO-READY SYSTEM BOUNDARY

Investor web app [React / TypeScript]

Screener; company research; peer comparisons; scenario views; saved shortlists and watchlists

Research operations console [Web app]

Exceptions; peer / model policy review; release control; freshness and coverage

HTTPS / JSON

HTTPS / JSON

Product API / application backend [Python / FastAPI]

Discover and screen companies | compare peers | evaluate models and scenarios | explain evidence | save criteria, shortlists and watchlists. Queries approved snapshots; validates commands; schedules research work.

identity checks

Identity provider [External system]

Sign-in and tokens. IPO-Ready enforces application roles and account permissions.

acquire versioned source data

read snapshots

enqueue research jobs

Data processing workers [Python]

Acquire and version evidence; parse facts and narrative; normalize and reconcile; route exceptions; produce approved fact snapshots.

jobs

Durable task queue [Job store / queue]

Research and refresh jobs; retries / dead letters; idempotency keys; outbox dispatch.

jobs

Intelligence workers [Python]

Financial / factor models; peer matching; valuation scenarios; ranking policies; cited research packaging. Heavy analysis runs asynchronously.

Disclosure + market systems [External]

Filing indexes; prospectus and peer disclosures; XML/XBRL/JSON financials; offer terms and dated market snapshots.

optional grounded calls

LLM provider [Optional external]

Tool-bound retrieval and explanations. Numbers, ranking policies and publication authority stay in the platform.

facts

SQL

objects

publish

Application DB [PostgreSQL]

Accounts, saved criteria, watchlists, reviews, jobs and release state

Research DB [PostgreSQL]

Point-in-time facts; features; peer sets; model registry; runs and lineage

Evidence store [Object storage]

Immutable filings, parsed text, model artifacts and research snapshots

Read models [PostgreSQL]

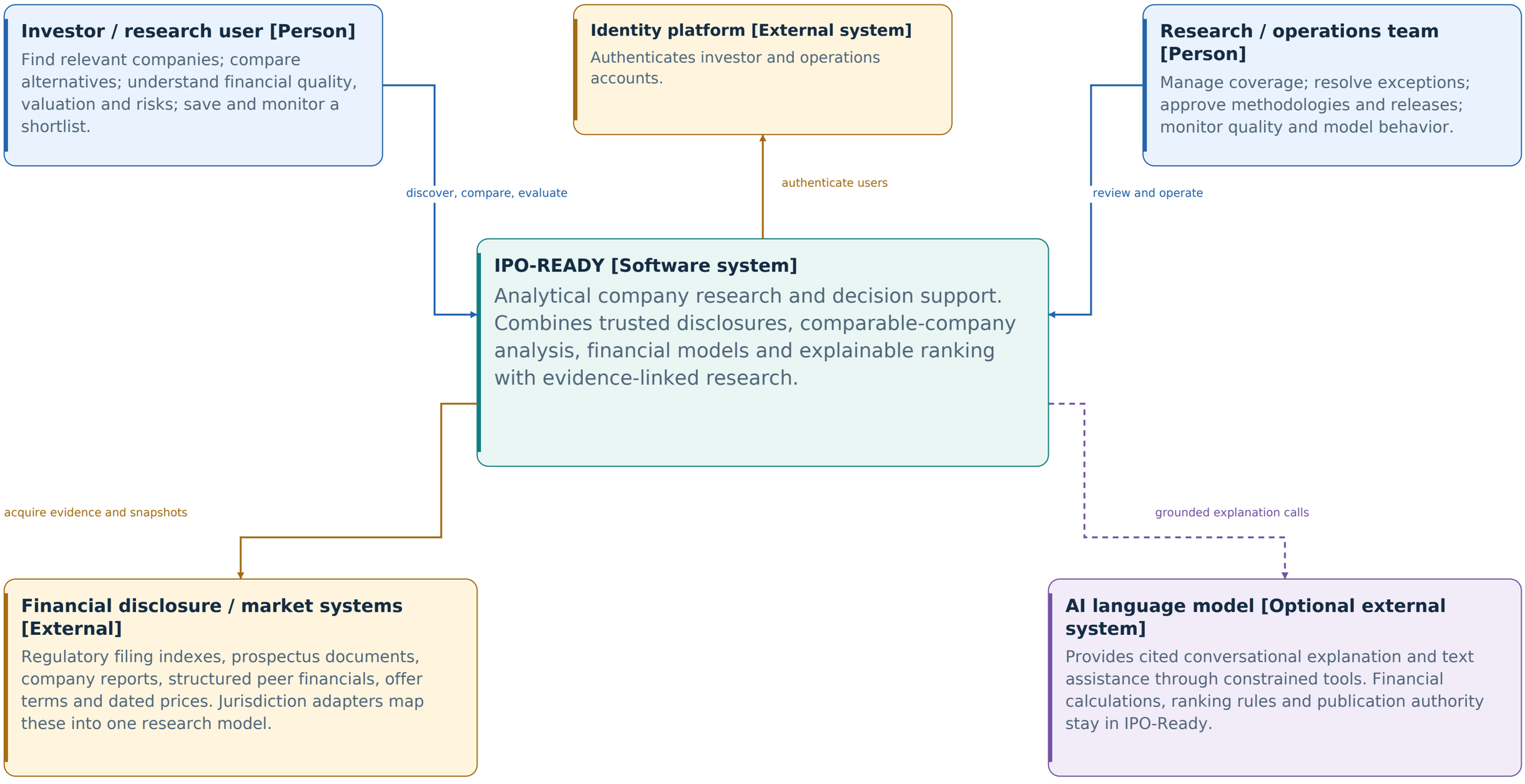
Published screener, company, comparison and scenario projections; optional cache

CROSS-CUTTING: ACCESS CONTROL, AUDIT, MODEL VERSIONS, OBSERVABILITY AND RECOVERY

Logical stores may share PostgreSQL initially. The backend is modular; data and intelligence workers can scale independently.

Product scope | System context

C4 system context view: the product is defined by investor decisions and external dependencies, independent of any prototype.



PRODUCT OUTCOME

A user can explain why a company is shortlisted, how it compares, what could change the view, and which evidence supports that view.

How analysis becomes useful company advice

Logical component view of the intelligence domain: explicit models and selection policies connect data to investor decisions.

DISCOVER
Filter the investment universe

→ **COMPARE**
Select meaningful alternatives

→ **EVALUATE**
Assess quality, risk and price

→ **SHORTLIST**
Apply explicit selection criteria

→ **MONITOR**
Track meaningful changes

USER-SELECTED RESEARCH CONTEXT

Sector / business model; market; offer status; horizon; metric filters; valuation scenarios; user weights where enabled. Store context with each result.

1. Eligible universe + peers

Resolve entity IDs. Apply disclosure/coverage gates. Match sector, business mix, scale and basis. Keep inclusion and exclusion reasons.

→

2. Financial feature engine

Growth; margins; cash conversion; leverage; liquidity; concentration. Standardize definitions and periods; preserve source lineage.

→

3. Model + scenario engine

Sector-specific quality/risk factors; relative valuation; explicit price and earnings scenarios. Version inputs, assumptions and policies.

→

4. Ranking + explanation

Rank eligible companies within a named cohort using a disclosed policy. Show factor contributions, trade-offs, missingness and counterarguments.

Selection-policy contract

Filters + cohort + factor definitions + weights + missing-data policy + tie handling + policy version. User-specific weights create a distinct saved research view.

Valuation / sensitivity contract

Compare price and enterprise/equity multiples on aligned dates and share bases. Display scenario ranges; unsupported methods stay unavailable. Forecast-based models are added only with explicit assumptions.

Explainability contract

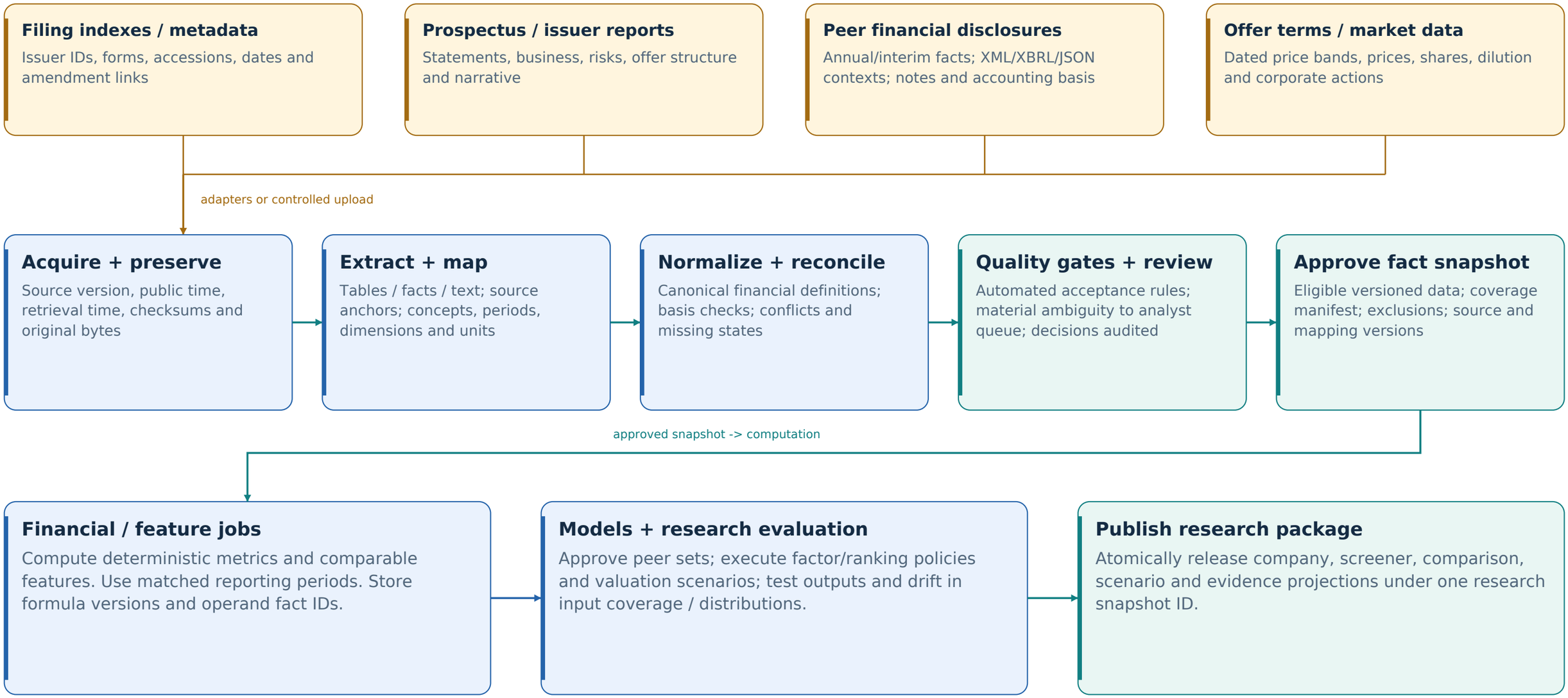
Display quality, financial risk, valuation and data coverage separately. A result links to factors, metrics, underlying facts and cited disclosures. AI explains these results; it does not invent the ranking.

MODEL LIFECYCLE

Define -> implement -> test on reviewed reference cases -> evaluate stability and coverage -> approve version -> monitor -> revise or roll back.

Research production | Data and model lifecycle

Asynchronous processing turns the listed data into reusable intelligence; user browsing reads prepared research products.



EXCEPTIONS DO NOT DISAPPEAR

Failed retrieval / extraction -> bounded retry or repair. Conflicting / ambiguous facts -> review. Incomplete metrics -> explicit unavailable states.

REFRESH LOOP

New filing, price refresh, correction or policy change -> identify dependencies -> recompute affected research -> release successor snapshot -> refresh watchlists.

Research state | Evidence, versions and consistency

Point-in-time data and frozen model inputs make a shortlist or comparison explainable and reproducible after the data changes.

DocumentVersion
Source identity; immutable hash; publication / retrieval time; supersedes relation

→

FactVersion
Raw + canonical value; metric; period / basis; evidence anchor; review / mapping version

→

MetricFeatureSet
Snapshot of derived metrics; formula versions; operand fact IDs; coverage states

→

ResearchRun
Fact / feature manifests; peer set; policy / model version; scenarios; code and run state

→

ResearchSnapshot
Approved outputs; cohort and assumptions; evidence; publication / expiry timestamps

Run manifest | the reproducibility contract

Pin source and fact versions; analysis cutoff; financial periods; market-data as-of times; currency/basis rules; peer membership; factor definitions and weights; model/scenario versions; code version; reviewer and release decision.

Serving contract | one coherent research version

All screener, company, comparison and explanation responses identify the research snapshot, coverage and relevant market-data time. Build projections before switching the published pointer. Cache keys include snapshot and query/criteria IDs.

RELIABLE CHANGE PROPAGATION

Commit change + outbox
One database transaction

→

Dispatch work event
Retry-safe event ID

→

Rebuild affected outputs
Idempotent worker / projector

→

Release + invalidate
Versioned pointer + cache event

TEMPORAL RULE
Source-publication time controls eligibility; retrieval time records when the platform learned it. Historical reconstruction is labelled. Later amendments do not rewrite older research.

History, audit records and outbox events are retained; full event sourcing is not required for this design.

Startup deployment | Reliability and growth

Deployment view: a practical launch topology, with scale boundaries chosen for actual workload rather than the prototype code.

Public edge + web delivery
HTTPS, CDN/static frontend delivery, request limits, web protection and authenticated sessions.

API

Private application runtime
Modular Python API; separate data and intelligence worker processes; scheduler/outbox dispatcher.

private

Managed data services
PostgreSQL schemas; durable jobs/queue; private object storage. Secrets and service identities scoped by role.

OPERATING CONTROLS APPLY TO EVERY DOMAIN

Access + data separation
Investor / analyst / admin roles; account-scoped saved research; common financial evidence kept separate from private user data.

Observability + freshness
Trace API request -> job -> research run; monitor lag, parser failures, coverage changes, model shifts and serving errors.

Recovery + controlled release
Database point-in-time recovery; evidence/artifact backups; restore drills; CI checks; staging; versioned model promotion and rollback.

LAUNCH NOW

Small, deliberate deployment
One modular backend; independently runnable workers; PostgreSQL facts/features/read projections; object storage; durable job table or managed queue. PostgreSQL full-text search first. AI is a replaceable provider adapter.

SCALE WHEN MEASUREMENTS JUSTIFY IT

Preserve the same domain contracts
Add worker replicas for backlog; dedicated cache/read replicas for query pressure; a search service for retrieval load; an analytical warehouse for large cohort scans. Extract a service only for measured isolation, scaling or ownership needs.

Proposed targets to validate: fast reads from published projections; explicit freshness labels; no duplicate publications; graceful use of the last valid snapshot.

Heavy extraction, model execution and optional AI inference remain off the synchronous browsing path.

Architecture decisions | Product delivery roadmap

The startup product is the design target. Repository mockups and attached reports supply context and examples, not architectural constraints.

1 / FOUNDATION

Define the intelligence contract

Company / offering identities; metric and factor policies; peer rules; evidence schema; representative reference cases.

2 / CORE PRODUCT

Ship discovery and comparison

Screener, company pages, peer comparison, explicit selection criteria, valuation scenarios, evidence and saved shortlists.

3 / AUTOMATION

Make research repeatable

Reliable acquisition / extraction; exception queue; model approval; published snapshot refresh; watchlist changes; cited assistant.

4 / EXPANSION

Grow coverage and capacity

Additional sectors / jurisdictions; evaluate new models; scale workers and read services using observed demand.

KEY ARCHITECTURE DECISIONS

Decision	Why it belongs in this product
C4 context + container + separate logical views	Keep users, runtime units, domain models and deployment concerns explicit.
Modular backend + asynchronous workers	Support domain separation and independent compute scaling without a service fleet.
Published read projections + immutable snapshots	Fast discovery/comparison; users see one coherent research version with freshness.
Versioned peer / factor / valuation policies	Selection and analytical advice can be explained, tested and revised.
Point-in-time facts + operand lineage	Reproduce research and prevent later disclosures from leaking into an earlier view.
Outbox + idempotent processing	Make source/model changes and downstream refresh recoverable under retries.

ARCHITECTURE REFERENCES / PRIMARY GUIDANCE

C4: c4model.com/diagrams/system-context and /container | Microsoft: Azure Architecture Center /patterns/cqrs

AWS Prescriptive Guidance: cloud-design-patterns/transactional-outbox.html | Accessed 29 Oct 2026

DELIVERY PRINCIPLE

Validate financial logic, ranking stability, evidence accuracy and user understanding before expanding analytical claims or product scope.